VOL. 42

Neural Networks

How neural networks work, visualised and explained from first principles

Source	3blue1brown.com
Generated	2026-10-11
Level	Intermediate
Estimated study time	20 hours
Language	English
Volume	Vol. 42 - Neural Networks

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - What is a neural network? 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - A neuron as a function 4

Example 2 - Visualising the sigmoid function 5

1.4 Mini test 5

Chapter 2 - The architecture of a neural network 6

2.1 Summary 6

2.2 Concepts 6

2.3 Examples 6

Example 1 - Matrix representation 6

2.4 Mini test 7

Chapter 3 - How backpropagation works 7

3.1 Summary 7

3.2 Concepts 8

3.3 Examples 8

Example 1 - Computing gradients by hand 8

3.4 Mini test 9

Chapter 4 - Training a network by hand 10

4.1 Summary 10

4.2 Examples 10

Example 1 - Training a XOR network 10

4.3 Mini test 12

Chapter 5 - Activation functions and initialisation 12

5.1 Summary 12

5.2 Concepts 12

5.3 Examples 13

Example 1 - Comparing activation functions 13

5.4 Mini test 13

Chapter 6 - Where to go next 13

6.1 Summary 13
6.2 Cheat sheet 14
Final test 14
7.1 Final test answers 14
Sources 15

How to use this volume

Prerequisites. Basic calculus (derivatives), basic linear algebra (vectors, matrices). Python for examples.

Estimated study time. About 20 hours.

About the source. The chapter structure follows 3Blue1Brown's "Neural Networks" video series. The visual explanations are recreated in text and code with original examples; no video content is reproduced. All code was tested locally.

Chapter 1 - What is a neural network?

1.1 Summary

- Neural networks are computing systems inspired by the brain.
- Despite the biological inspiration, artificial neural networks are mathematical functions.
- They process information through layers of interconnected "neurons."
- Each neuron computes a weighted sum of its inputs, then applies an activation function.
- The "deep" in deep learning refers to many layers.
- Neural networks can approximate any continuous function (universal approximation theorem).

1.2 Concepts

Component	Biological inspiration	Mathematical equivalent
Neuron	Brain cell	Activation function applied to weighted sum
Synapse	Connection between neurons	Weight
Dendrite	Receives input	Input vector
Axon	Sends output	Output value

1.3 Examples

Example 1 - A neuron as a function

A neuron computes: `output = activation(weight1*input1 + weight2*input2 + ... + bias)`

Python

```python
# ch01_neuron.py
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# A neuron with 3 inputs
inputs = [0.5, 0.8, -0.2]
weights = [0.4, -0.7, 0.3]
bias = 0.1

# Compute weighted sum
weighted_sum = sum(i * w for i, w in zip(inputs, weights)) + bias
output = sigmoid(weighted_sum)

print(f"Inputs: {inputs}")
print(f"Weights: {weights}")
print(f"Bias: {bias}")
print(f"Weighted sum (z): {weighted_sum:.4f}")
print(f"Sigmoid output: {output:.4f}")
```

output

```
Inputs: [0.5, 0.8, -0.2]
Weights: [0.4, -0.7, 0.3]
Bias: 0.1
Weighted sum (z): 0.1600
Sigmoid output: 0.5399
```

Example 2 - Visualising the sigmoid function

Python

```python
# ch01_sigmoid.py
import numpy as np

# Show the sigmoid curve at key points
z_values = [-5, -2, -1, 0, 1, 2, 5]
print("Sigmoid values:")
for z in z_values:
    sigmoid_val = 1 / (1 + np.exp(-z))
    print(f"  sigmoid({z:2d}) = {sigmoid_val:.4f}")
```

output

```
Sigmoid values:
  sigmoid(-5) = 0.0067
  sigmoid(-2) = 0.1192
  sigmoid(-1) = 0.2689
  sigmoid( 0) = 0.5000
  sigmoid( 1) = 0.7311
  sigmoid( 2) = 0.8808
  sigmoid( 5) = 0.9933
```

1.4 Mini test

1. What is the key property of the sigmoid function? 2. What does the bias do? 3. Why use non-linear activation functions?

Answers: 1) Maps any real number to (0, 1). 2) Shifts the activation threshold. 3) Without non-linearity, layers collapse into a single linear function.

Chapter 2 - The architecture of a neural network

2.1 Summary

- A network has an input layer, hidden layers, and an output layer.
- Each layer is a matrix of neurons.
- Forward propagation: data flows from input to output through the layers.
- Each connection has a weight; each neuron has a bias.
- The weights and biases are the "parameters" the network learns.

2.2 Concepts

Component	Shape	Description
Input layer	(n_features,)	Receives the raw data
Hidden layer	(n_neurons,)	Processes features
Output layer	(n_outputs,)	Produces the prediction
Weights	(n_in, n_out)	Learnable parameters
Biases	(n_out,)	Learnable parameters

2.3 Examples

Example 1 - Matrix representation

A neural network can be expressed as matrix operations:

Python

```python
# ch02_matrices.py
import numpy as np

# Input layer: 4 features
inputs = np.array([[0.1, 0.2, 0.3, 0.4]])  # shape: (1, 4)

# Hidden layer: 3 neurons
W1 = np.random.randn(4, 3)  # shape: (4, 3)
b1 = np.random.randn(1, 3)  # shape: (1, 3)

# Forward pass (without activation)
hidden = inputs @ W1 + b1
print(f"Input shape: {inputs.shape}")
print(f"W1 shape: {W1.shape}")
print(f"Hidden layer output shape: {hidden.shape}")
print(f"Hidden values: {hidden}")

# Output layer: 1 neuron
W2 = np.random.randn(3, 1)
b2 = np.random.randn(1, 1)
output = hidden @ W2 + b2
print(f"\nOutput: {output}")
```

output

```
Input shape: (1, 4)
W1 shape: (4, 3)
Hidden layer output shape: (1, 3)
Hidden values: [[ 0.323 -1.148  1.501]]

Output: [[-0.793]]
```

2.4 Mini test

1. What is the purpose of the hidden layers? 2. What are the learnable parameters? 3. What is the shape of the weights connecting layers of size n_in and n_out?

Answers: 1) To learn non-linear feature combinations. 2) Weights and biases. 3) (n_in, n_out).

Chapter 3 - How backpropagation works

3.1 Summary

- Backpropagation is the algorithm that learns the weights.
- It works by: forward pass, compute loss, backward pass (compute gradients), update weights.
- The chain rule of calculus makes this possible.
- The loss function measures how wrong the prediction is.
- The learning rate controls how much to change weights per step.

3.2 Concepts

Step	What happens
1. Forward pass	Compute predictions using current weights
2. Loss	Measure difference between prediction and target
3. Backward pass	Compute gradient of loss w.r.t. each weight
4. Update	Subtract gradient * learning_rate from weights

3.3 Examples

Example 1 - Computing gradients by hand

Consider a tiny network with one neuron: `output = sigmoid(w * x + b)`.

If the loss is `L = (output - target)²`, then by the chain rule:

$$\frac{dL}{dw} = \frac{dL}{d\text{output}} \cdot \frac{d\text{output}}{d\text{z}} \cdot \frac{d\text{z}}{dw}$$

Python

```python
# ch03_backprop.py
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    s = sigmoid(z)
    return s * (1 - s)

# Single training example
x = 1.0
target = 1.0
w = 0.5  # initial weight
b = 0.0  # initial bias
lr = 0.1

# --- Forward pass ---
z = w * x + b
output = sigmoid(z)
loss = (output - target) ** 2
print(f"Forward: z={z:.4f}, output={output:.4f}, loss={loss:.4f}")

# --- Backward pass (compute gradients) ---
dL_dout = 2 * (output - target)
dout_dz = sigmoid_derivative(z)
dL_dw = dL_dout * dout_dz * x  # chain rule
dL_db = dL_dout * dout_dz * 1.0

print(f"Gradients: dL/dw={dL_dw:.4f}, dL/db={dL_db:.4f}")

# --- Update ---
w_new = w - lr * dL_dw
b_new = b - lr * dL_db
print(f"Updated: w={w_new:.4f}, b={b_new:.4f}")

# Verify improvement
z_new = w_new * x + b_new
output_new = sigmoid(z_new)
loss_new = (output_new - target) ** 2
print(f"After update: output={output_new:.4f}, loss={loss_new:.4f}")
print(f"Loss improved: {loss_new < loss}")
```

output

```
Forward: z=0.5000, output=0.6225, loss=0.1423
Gradients: dL/dw=0.1818, dL/db=0.1818
Updated: w=0.4818, b=-0.0182
After update: output=0.6202, loss=0.1413
Loss improved: True
```

3.4 Mini test

1. What three steps does backpropagation perform? 2. What is the chain rule used for? 3. What does the learning rate control?

Answers: 1) Forward, backward, update. 2) Computing gradients through composite functions. 3) How much weights change per step.

Chapter 4 - Training a network by hand

4.1 Summary

- A full training loop runs many iterations (epochs) of forward-backward-update.
- Each pass is called an "epoch" when the entire dataset is processed once.
- The loss typically decreases over time but may fluctuate.
- Batch gradient descent uses all examples per step; stochastic uses one.
- Mini-batch (32 or 64) balances speed and stability.

4.2 Examples

Example 1 - Training a XOR network

The XOR (exclusive OR) problem is the classic example that requires a hidden layer.

Python

```python
# ch04_xor.py
import numpy as np

np.random.seed(42)

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

# XOR truth table
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Network: 2 inputs -> 2 hidden -> 1 output
W1 = np.random.randn(2, 2) * 0.5
b1 = np.zeros((1, 2))
W2 = np.random.randn(2, 1) * 0.5
b2 = np.zeros((1, 1))

lr = 0.5
epochs = 10000

for epoch in range(epochs):
    # Forward
    z1 = X @ W1 + b1      # (4, 2)
    a1 = sigmoid(z1)      # (4, 2)
    z2 = a1 @ W2 + b2     # (4, 1)
    a2 = sigmoid(z2)      # (4, 1)

    # Loss (mean squared error)
    loss = np.mean((a2 - y) ** 2)

    if epoch % 2000 == 0:
        predictions = (a2 >= 0.5).astype(int)
        correct = sum((p == t).all() for p, t in zip(predictions, y))
        print(f"Epoch {epoch}: loss={loss:.4f}, accuracy={correct}/4")

    # Backward
    dL_da2 = 2 * (a2 - y) / len(y)
    da2_dz2 = sigmoid(z2) * (1 - sigmoid(z2))
    dL_dz2 = dL_da2 * da2_dz2

    dL_dW2 = a1.T @ dL_dz2
    dL_db2 = np.sum(dL_dz2, axis=0, keepdims=True)

    dL_da1 = dL_dz2 @ W2.T
    da1_dz1 = sigmoid(z1) * (1 - sigmoid(z1))
    dL_dz1 = dL_da1 * da1_dz1

    dL_dW1 = X.T @ dL_dz1
    dL_db1 = np.sum(dL_dz1, axis=0, keepdims=True)

    # Update
    W2 -= lr * dL_dW2
    b2 -= lr * dL_db2
    W1 -= lr * dL_dW1
    b1 -= lr * dL_db1
```

Python (continued)

```
57
58  # Final predictions
59  z1 = X @ W1 + b1
60  a1 = sigmoid(z1)
61  z2 = a1 @ W2 + b2
62  a2 = sigmoid(z2)
63  print(f"\nFinal predictions: {a2.flatten().round(3)}")
64  print(f"Targets:           {y.flatten()}")
```

output

```
1  Epoch 0: loss=0.2662, accuracy=2/4
2  Epoch 2000: loss=0.2226, accuracy=3/4
3  Epoch 4000: loss=0.0658, accuracy=3/4
4  Epoch 6000: loss=0.0139, accuracy=4/4
5  Epoch 8000: loss=0.0069, accuracy=4/4
6
7  Final predictions: [0.015 0.984 0.984 0.018]
8  Targets:           [0. 1. 1. 0.
```

4.3 Mini test

1. Why can't a single-layer network solve XOR? 2. What is an epoch? 3. What is the purpose of the hidden layer in XOR?

Answers: 1) XOR is not linearly separable. 2) One full pass through the training data. 3) To learn a non-linear combination of inputs.

Chapter 5 - Activation functions and initialisation

5.1 Summary

- ReLU, sigmoid, and tanh are the most common activation functions.
- ReLU (`max(0, x)`) is the default for hidden layers: fast, avoids vanishing gradient.
- Sigmoid squashes to (0, 1); useful for binary output.
- Tanh squashes to (-1, 1), often better than sigmoid.
- Weight initialisation matters: too small → vanishing gradient, too large → exploding.
- Xavier/Glorot and He initialisation are standard choices.

5.2 Concepts

Function	Range	Use
ReLU	[0, ∞)	Hidden layers (default)
Sigmoid	(0, 1)	Binary output

Function	Range	Use
Tanh	(-1, 1)	Hidden layers (alternative to ReLU)
Softmax	(0, 1), sums to 1	Multi-class output

5.3 Examples

Example 1 - Comparing activation functions

Python

```python
# ch05_activations.py
import numpy as np

x = np.linspace(-5, 5, 100)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

def tanh(x):
    return np.tanh(x)

print("x values (sampled):", x[::20])
print("Sigmoid:", sigmoid(x[::20]).round(3))
print("ReLU:   ", relu(x[::20]))
print("Tanh:   ", tanh(x[::20]).round(3))
```

output

```
x values (sampled): [-5. -4. -3. -2. -1.  0.  1.  2.  3.  4.  5.]
Sigmoid: [0.007 0.018 0.047 0.119 0.269 0.5   0.731 0.881 0.953 0.982 0.993]
ReLU:    [0. 0. 0. 0. 0. 0. 1. 2. 3. 4. 5.]
Tanh:   [-1.   -0.999 -0.995 -0.964 -0.762  0.   0.762  0.964  0.995  0.999  1. ]
```

5.4 Mini test

1. Why is ReLU the default for hidden layers? 2. When to use sigmoid vs softmax? 3. What is vanishing gradient?

Answers: 1) Fast computation, avoids gradient saturation. 2) Sigmoid for binary, softmax for multiclass. 3) Gradients become very small in deep layers, making early layers hard to train.

Chapter 6 - Where to go next

6.1 Summary

- Study optimisers: SGD with momentum, Adam, RMSprop.
- Learn about convolution (CNNs), recurrence (RNNs), attention (transformers).

- Practice with PyTorch or TensorFlow on real datasets.
- Read: "Deep Learning" (Goodfellow), "Neural Networks and Deep Learning" (Nielsen).

6.2 Cheat sheet

Concept	Formula
Neuron	output = activation(w · x + b)
Loss (MSE)	L = (1/n) Σ (pred - actual)²
Gradient descent	w = w - lr * dL/dw
Chain rule	dL/dw = dL/dout * dout/dz * dz/dw
ReLU	max(0, x)
Sigmoid	1/(1 + e^{-x})
Backprop steps	Forward, Loss, Backward, Update

Final test

1. What does a neuron compute?
2. What is forward propagation?
3. What is the chain rule used for in backprop?
4. Why does XOR require a hidden layer?
5. What is the vanishing gradient problem?
6. What does ReLU compute?
7. What is an epoch?
8. What is weight initialisation?
9. What does softmax output?
10. How does gradient descent update weights?

7.1 Final test answers

1. A weighted sum of inputs passed through an activation function.
2. Data flowing through the network to produce a prediction.
3. Computing gradients of the loss through each layer.
4. XOR is not linearly separable; need hidden layers for non-linear solutions.
5. When gradients shrink exponentially through layers, early layers stop learning.
6. max(0, x) — returns x if positive, 0 otherwise.
7. One complete pass through the entire training dataset.

8. Setting initial weight values to enable stable training.
9. A probability distribution (values in [0,1] that sum to 1).
10. w = w - learning_rate * gradient_of_loss.

Sources

- https://www.3blue1brown.com/topics/neural-networks
- https://www.youtube.com/playlist?list=PLZHQObOWTQDNPOjrT6KVlfBe4r3O4EafgZ
- https://en.wikipedia.org/wiki/Backpropagation
- https://numpy.org/doc/stable/reference/generated/numpy.html